

goPrivate MVP - Technical Specification for Cursor

Project Overview

Build the MVP of **goPrivate**, a browser-based ephemeral messaging platform.

The MVP should allow two users to create a private chat session without creating accounts.

Messages are end-to-end encrypted.

The relay server never has access to plaintext messages.

The relay server stores messages only in memory until delivery.

There is no database.

There are no user accounts.

There are no contacts.

There is no message history.

This MVP is intended to validate the core user experience, not solve every privacy edge case.

Core Principles

1. Privacy by Design

The server must never know:

- User identity
- User profile
- Contact list
- Message content
- Conversation history

The relay only forwards encrypted packets.

2. Ephemeral Everything

The system should avoid persistence wherever possible.

No SQL database.

No MongoDB.

No Redis.

No user storage.

No analytics.

Everything exists only while the session exists.

3. Simplicity

Build the smallest system that demonstrates:

- Create session
- Share link
- Join session
- Exchange encrypted messages
- Disconnect

Nothing else.

Technology Stack

Frontend

- Next.js (App Router)
 - React
 - TypeScript
 - TailwindCSS
 - shadcn/ui
 - Zustand
-

Backend

- Node.js
 - Fastify
 - WebSockets
-

Crypto

Use browser-native Web Crypto API.

Do NOT implement custom cryptography.

Create a CryptoService abstraction so implementations can evolve later.

Deployment

Docker

Single relay server

No Kubernetes

No microservices

Repository Structure

```
goprivate/  
  
  apps/  
    web/  
    relay/  
  
  packages/  
    protocol/  
    crypto/  
    sdk/  
  
  docs/
```

Browser Application

Pages

Landing

Route

/

Contains

- Hero
 - Create Session button
 - Join Session input
-

Chat

/chat/[sessionId]

Contains

- Message list
- Message input
- Send button
- Leave button

No contact list.

No sidebar.

No settings.

Session Flow

User A

Clicks

Create Session

Browser

- Generates random Session ID
- Generates encryption key pair
- Opens WebSocket
- Registers session with relay

Returns

```
https://domain/chat/SESSION_ID
```

User B

Opens link.

Browser

- Opens WebSocket
- Joins session

Handshake begins.

Relay Responsibilities

The relay should only

- Accept websocket connections
- Associate sockets with Session ID
- Forward encrypted packets
- Remove disconnected sockets

Nothing else.

Never inspect payload.

Never decrypt.

Never persist.

In-Memory Storage

Use

```
Map<SessionID, Session>
```

Example

```
Session  
  
id  
  
participants[]  
  
socketA  
  
socketB  
  
createdAt
```

When both sockets disconnect

Destroy session.

WebSocket Events

Client → Relay

```
CREATE_SESSION  
  
JOIN_SESSION  
  
SEND_MESSAGE  
  
PING  
  
LEAVE_SESSION
```

Relay → Client

SESSION_CREATED

PARTNER_JOINED

MESSAGE

PARTNER_LEFT

ERROR

Message Object

```
{  
  id,  
  encryptedPayload,  
  timestamp  
}
```

The relay treats encryptedPayload as an opaque string.

Encryption

Before first message

Generate

ECDH key pair

Exchange

Public Keys

Derive

Shared Secret

Encrypt

AES-GCM

Every outgoing message

Plaintext

↓

Encrypt

↓

Ciphertext

↓

Relay

↓

Recipient

↓

Decrypt

State Management

Zustand store

connection

session

messages

peer

status

Components

LandingPage

CreateSessionButton

JoinSessionForm

ChatPage

MessageList

MessageBubble

MessageComposer

ConnectionStatus

Header

UI

Very minimal.

Inspired by

Signal

Linear

Notion

Use lots of whitespace.

No gradients.

No animations except subtle fades.

MVP Features

Must Have

- ✓ Create session
 - ✓ Join session
 - ✓ WebSocket connection
 - ✓ End-to-end encrypted text messages
 - ✓ Leave session
 - ✓ Session destroyed when everyone leaves
-

Not Included

- ✗ Authentication
- ✗ Accounts
- ✗ Contacts
- ✗ Database
- ✗ Images
- ✗ Files
- ✗ Voice
- ✗ Video
- ✗ Read receipts
- ✗ Typing indicators
- ✗ Push notifications
- ✗ Message persistence

Folder Structure

```
apps/  
  web/  
    app/  
    components/  
    hooks/  
    services/  
    store/  
    utils/  
  relay/  
    src/  
      websocket/  
      session/  
      handlers/  
      services/  
  packages/  
    protocol/  
    crypto/  
    sdk/
```

Coding Standards

- Strict TypeScript
 - ESLint
 - Prettier
 - Functional React components
 - No class components
 - Dependency injection where practical
 - Small focused modules
 - No duplicated logic
-

Future Architecture

Design interfaces now for future expansion.

Examples

```
ICryptoProvider
```

```
IRelayClient
```

```
ISessionStore
```

```
ITransport
```

Current implementation may be simple, but should allow future replacement.

Development Phases

Phase 1

Project setup

Next.js

Fastify

Docker

Phase 2

Relay

WebSockets

Session creation

Join session

Phase 3

Encryption

ECDH

AES-GCM

Secure message exchange

Phase 4

Chat UI

Realtime messages

Leave session

Reconnect handling

Phase 5

Cleanup

Destroy empty sessions

Error handling

Testing

Success Criteria

The MVP is complete when the following workflow succeeds:

1. User A opens the website.
2. User A clicks **Create Session**.
3. A unique shareable link is generated.
4. User B opens the link in another browser.
5. Both browsers establish an encrypted connection.
6. User A sends a message.
7. User B receives and decrypts it.
8. User B replies successfully.
9. Both users leave the chat.
10. The relay destroys the session.
11. No message data remains on the server.

If these eleven steps work reliably, the MVP is complete.

Do not add additional features until this workflow is stable.